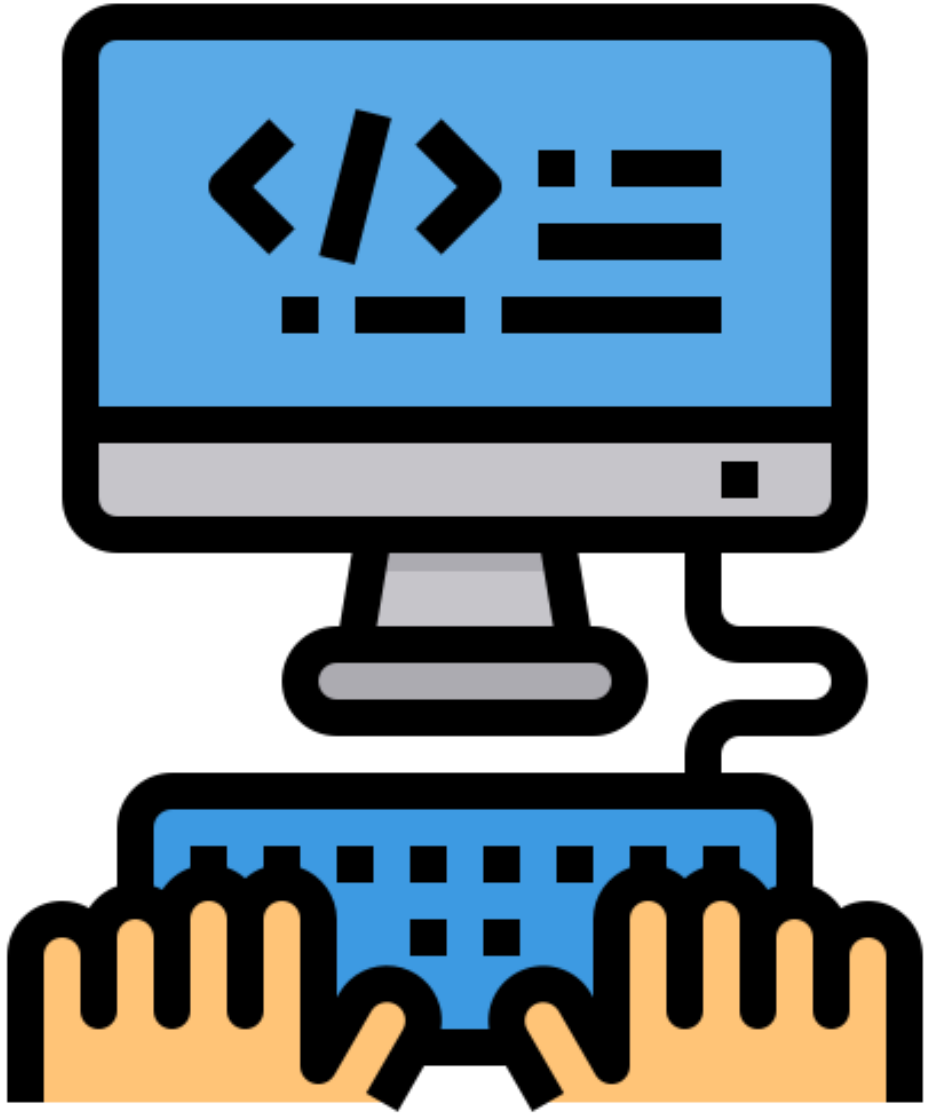
CS 24 AP Computer Science A Review

Week 9: Recursion

DR. ERIC CHOU
IEEE SENIOR MEMBER

Topics

- What is recursion?
- Call Stack and Binding-Environment
- Recursion to Iteration and Iteration to Recursion
- Recursion with Index, Accumulator
- Tail Recursion
- Complexity Analysis



What is Recursion?

SECTION 1

What is recursion?

- A function calling itself is called recursion.
- A recursion function has three major parts: base case, processing state and recursive calls.
- The base case (also named as termination case, exit case)
- The processing stage is used to prepare the data for proper recursive calls.
- The recursive calls are to call the same function recursively.

Steps to write a Recursive Program

1. The Termination Condition (Base Case)
2. Recursive Formula (Recursive Call)
3. Step by Step Improvement

Sum of 1 to n

```java
public class SumForward
{
    public static int Sum(int n){
        if (n == 1) return 1;

    }
}
```

(1) Base Case

```java
public class SumForward
{
    public static int Sum(int n){
        if (n == 1) return 1;
        return Sum(n-1) + n ;
    }
}
```

(2) Recursive Formula

```java
public class SumForward
{
    public static int Sum(int n){
        if (n == 1) return 1;    // base case
        return Sum(n-1) + n ;    // recursive formula
    }
    public static void main(String[] args){
        System.out.println(Sum(10));
    }
}
```

(3) Test Setup

BlueJ: Te...

Options

55

Can only enter inpu

Sum of 1 to n

Helper Method with Accumulator (For tail-recursive methods)

```java
public class SumBackward{
    public static int Sum(int n, int sum){
        if (n==0) return sum;
        return Sum(n-1, sum+n);
    }
}
```

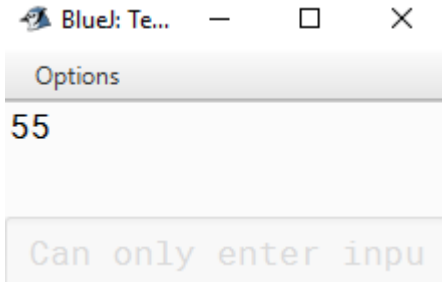
(1) Base Case

```java
public class SumBackward{
    public static int Sum(int n, int sum){
        if (n==0) return sum;
        return Sum(n-1, sum+n);
    }
    public static void main(String[] args){
        System.out.println(Sum(10, 0));
    }
}
```

(2) Recursive Formula

```java
public class SumBackward{
    public static int Sum(int n, int sum){
        if (n==0) return sum;
        return Sum(n-1, sum+n);
    }
    public static int Sum(int n){ return Sum(n, 0); }
    public static void main(String[] args){
        System.out.println(Sum(10));
    }
}
```

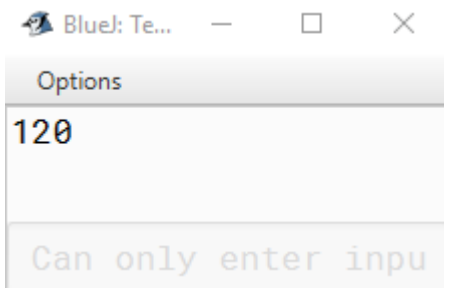
(3) Test Case with Normal Method



Factorial

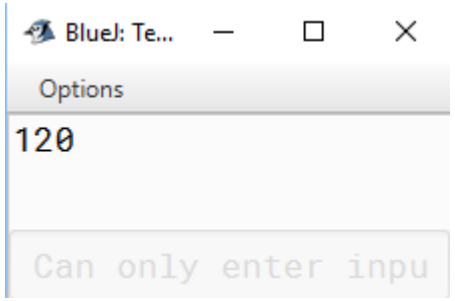
```java
public class Factorial{
    public static int f(int n){
      if (n==0) return 1;
      return f(n-1)*n;
    }
    public static void main(String[] args){
        System.out.println(f(5));
    }
}
```

(1) Normal Version




```java
public class FactorialHelper{
    public static int f_helper(int n, int product){
      if (n==0) return product;
      return f_helper(n-1, product * n);
    }
    public static int f(int n){ return f_helper(5, 1);}
    public static void main(String[] args){
        System.out.println(f(5));
    }
}
```

(2) Tail-recursive Version



Multiple Choice (1)

Consider the following method:

```
public int mystery(int x){
   if (x==0){
      return 0;
   }
   else{
      return ((x%10) + mystery(1/10));
   }
}
```

Which of the following is returned as a result of the call mystery(3543)?

Multiple Choice (1)

Ⓐ 15

Ⓑ 22

Ⓒ 180

Ⓓ Nothing is returned due to the infinite recursion.

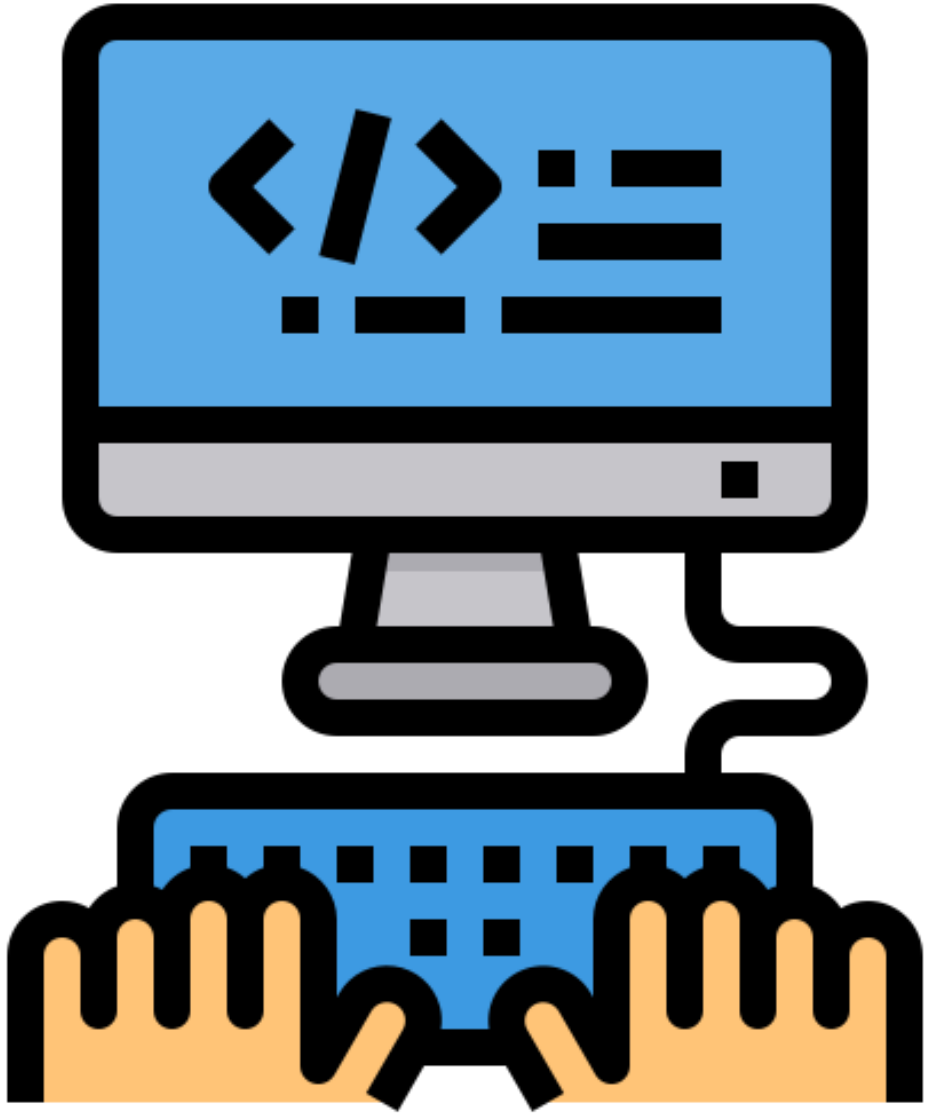
Multiple Choice (1)

Ⓐ 15

Ⓑ 22

Ⓒ 180

Ⓓ Nothing is returned due to the infinite recursion.



Iteration to Recursion

SECTION 2

Conversion of Sum to Iterative

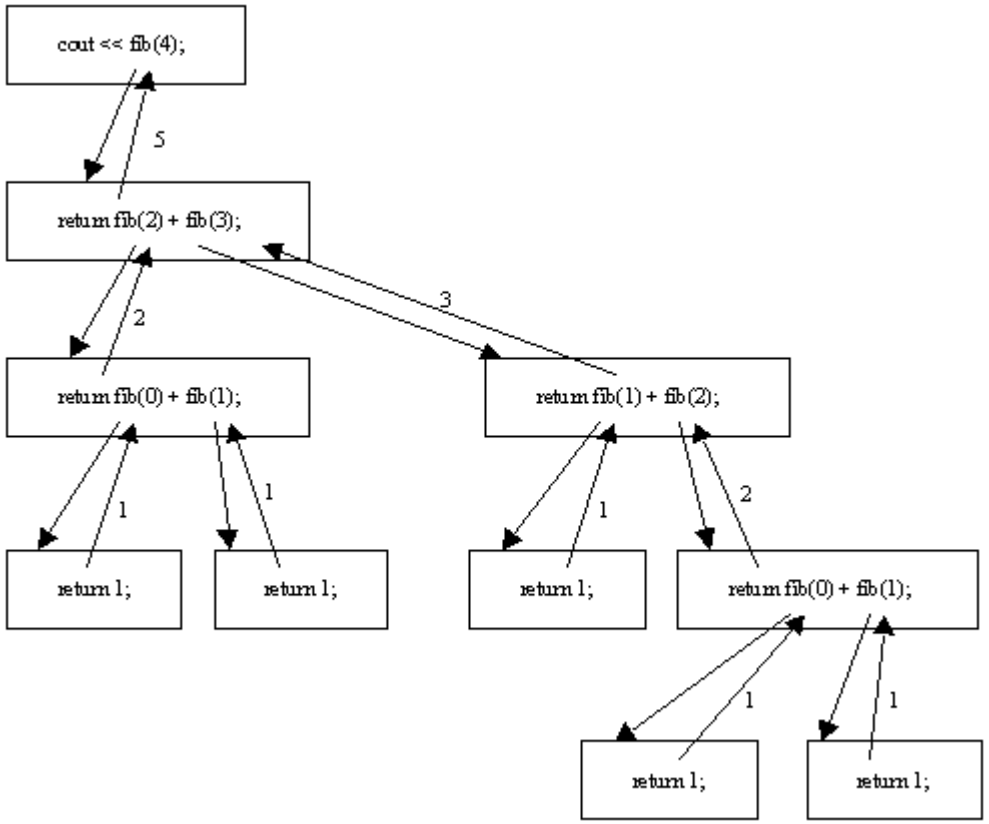
```
public static int Sum(int n){
    if (n == 1) return 1;   // base case
    return Sum(n-1) + n ;   // recursive formula
}
```

Base condition

Recursive Formula

```
public static int Sum(int n){
  int s = 0;
  for (int i=0; i<=n; i++){
    s = s + i;
  }
  return s;
}
```

Tracing Recursive Functions



Recursive Versus Iterative

```
static int fibonacciI(int n){
 int[] a = new int[n+1];
 a[0] = 1;
 a[1] = 1;
 for (int i = 2; i<=n; i++){
    a[i] = a[i-1] + a[i-2];
  }
 return a[n];
} // runs faster
static int fibonacciR(int n){
if (n == 0 || n == 1) return 1;
return fibonacciR(n-1) + fibonacciR(n-2);
} // shorter code
```

Base Case

Recursive Formula

```
n=21  Iterative Time: 0  Recursive Time: 0   Difference: 0
n=22  Iterative Time: 0  Recursive Time: 1   Difference: 1
n=23  Iterative Time: 0  Recursive Time: 0   Difference: 0
n=24  Iterative Time: 0  Recursive Time: 0   Difference: 0
n=25  Iterative Time: 0  Recursive Time: 0   Difference: 0
n=26  Iterative Time: 0  Recursive Time: 1   Difference: 1
n=27  Iterative Time: 0  Recursive Time: 1   Difference: 1
n=28  Iterative Time: 0  Recursive Time: 2   Difference: 2
n=29  Iterative Time: 0  Recursive Time: 2   Difference: 2
n=30  Iterative Time: 0  Recursive Time: 4   Difference: 4
n=31  Iterative Time: 0  Recursive Time: 6   Difference: 6
n=32  Iterative Time: 0  Recursive Time: 10   Difference: 10
n=33  Iterative Time: 0  Recursive Time: 18   Difference: 18
n=34  Iterative Time: 0  Recursive Time: 32   Difference: 32
n=35  Iterative Time: 0  Recursive Time: 42   Difference: 42
n=36  Iterative Time: 0  Recursive Time: 75   Difference: 75
n=37  Iterative Time: 0  Recursive Time: 115   Difference: 115
n=38  Iterative Time: 0  Recursive Time: 181   Difference: 181
n=39  Iterative Time: 0  Recursive Time: 321   Difference: 321
n=40  Iterative Time: 0  Recursive Time: 481   Difference: 481
```

Tracing Recursive Functions

- Write function calls one by one.
- Write function name in a letter if possible.
- Be aware of what need to be added and what need to be returned.
- Trace them step by step.

39. Consider the following recursive method.

```
public int recur(int n)
{
  if (n <= 10)
    return n * 2;
  else
    return recur(recur(n / 3));
}
```

What value is returned as a result of the call `recur(27)` ?

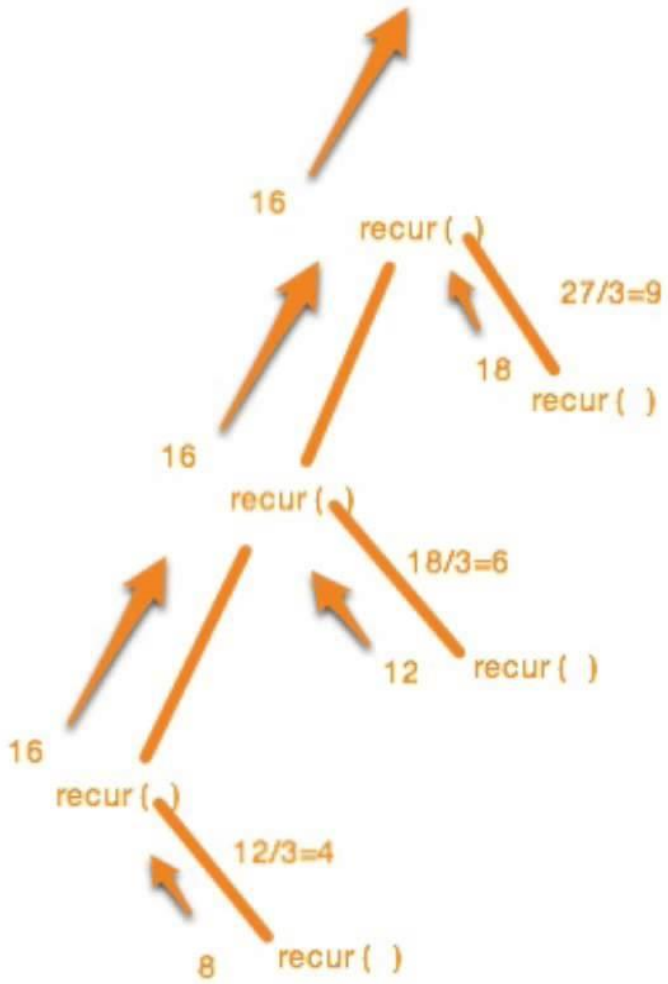
(A) 8

(B) 9

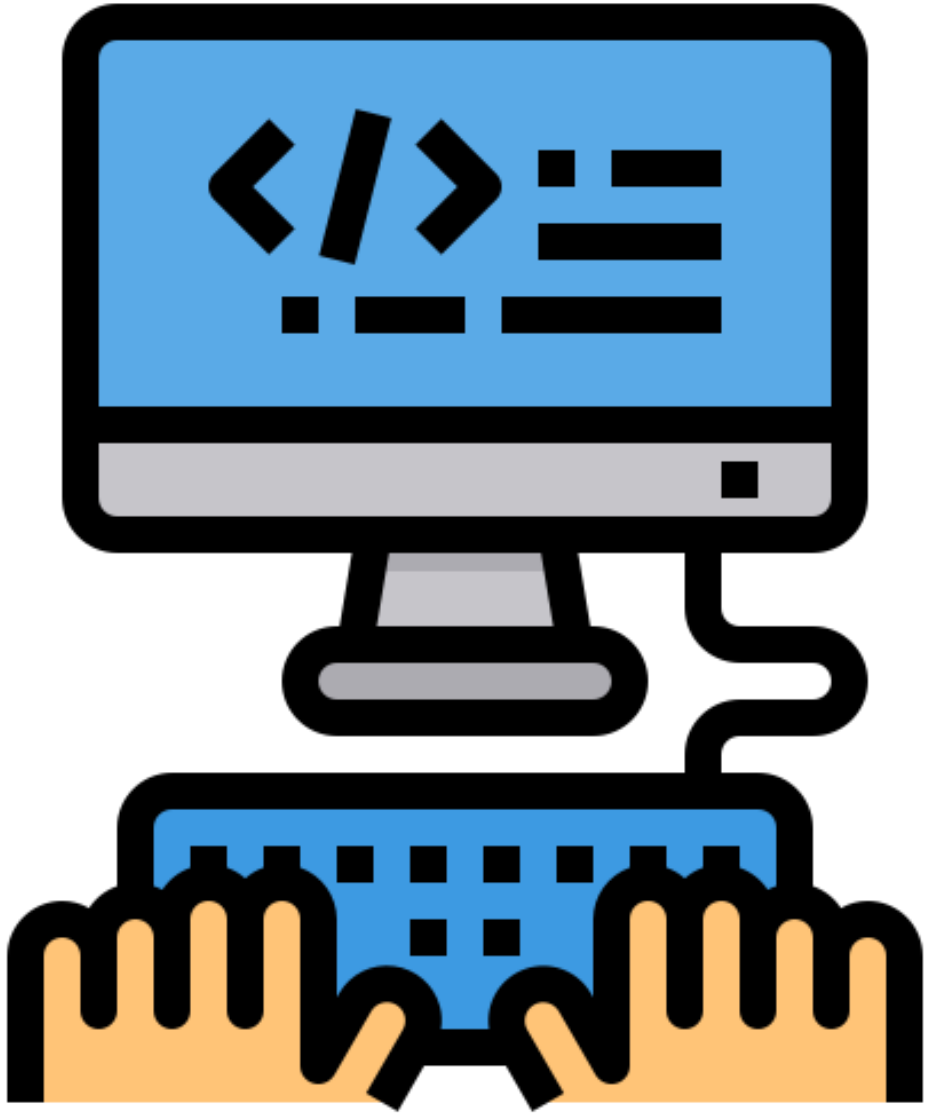
(C) 12

(D) 16

(E) 18



Write down the recursive steps one by one like a robot!!!
recur(27)-> recur(recur(9))->
recur(18)->recur(recur(6))->
recur(12)-> recur(recur(4))->
recur(8) -> return 16



Recursion with Index and Accumulator

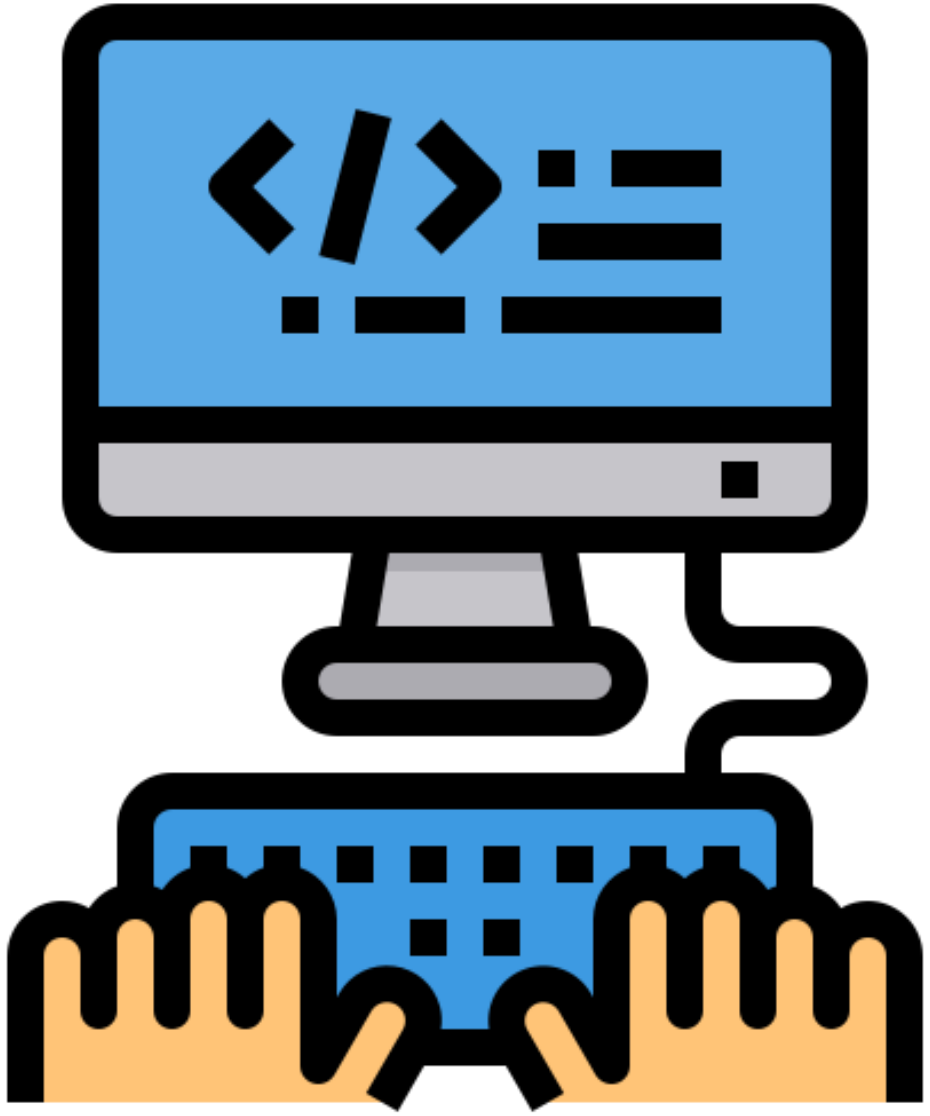
SECTION 3

```java
public class RunningIndex
{
    static int[] a = {1, 2, 3, 4, 5};

    public static int sum(int[] a){
        return sum(a, 0);
    }
    public static int sum(int[] a, int idx){
        if (idx >=a.length) return 0;
        return sum(a, idx+1)+a[idx];
    }

    public static void main(String[] args){
        System.out.println(sum(a));
    }
}
```

Running Index Recursion

```java
public class Accumulator
{
    static int[] a= {1, 2, 3, 4,5};

    public static int sum(int[] a){
        return sum(a, 0, 0);
    }

    public static int sum(int[] a, int idx, int s){
      if (idx >=a.length) return s;
      return sum(a, idx+1, s+a[idx]);
    }

    public static void main(String[] args){
        System.out.println(sum(a));
    }
}
```

Recursion with Accumulator



Recursive Function Over Array

Section 4

Palindrome Examples

- A palindrome is a word, phrase, number or sequence of words that reads the same backwards as forwards. Punctuation and spaces between the words or lettering is allowed (or not in CS).

Single Word Palindromes

- Anna
- Civic
- Kayak
- Level
- Madam
- Mom
- Noon
- Racecar
- Radar
- Redder
- Refer
- Repaper
- Rotator
- Rotor
- Sagas
- Solos
- Stats
- Tenet
- Wow

Recursive Palindrome

- Recursively check the string's characters layer by layer. If two characters of the same layer are matched, then continue to check otherwise reporting false.
- If no characters (or one characters left) then declare that the input string is indeed a palindrome word.

```java
public class RecursivePalindrome {
  public static boolean isPalindrome(String s) {
    return isPalindrome(s, 0, s.length() - 1);
  }

  private static boolean isPalindrome(String s, int low, int high) {
    if (high <= low) // Base case
      return true;
    else if (s.charAt(low) != s.charAt(high)) // Base case
      return false;
    else
      return isPalindrome(s, low + 1, high - 1);
  }

  public static void main(String[] args) {
    System.out.println("Is moon a palindrome? "
      + isPalindrome("moon"));
    System.out.println("Is noon a palindrome? "
      + isPalindrome("noon"));
    System.out.println("Is a a palindrome? " + isPalindrome("a"));
    System.out.println("Is aba a palindrome? " +
      isPalindrome("aba"));
    System.out.println("Is ab a palindrome? " + isPalindrome("ab"));
  }
}
```

Adding a Helper to Remove Unrelated Characters and Allow Case-Insensitiveness

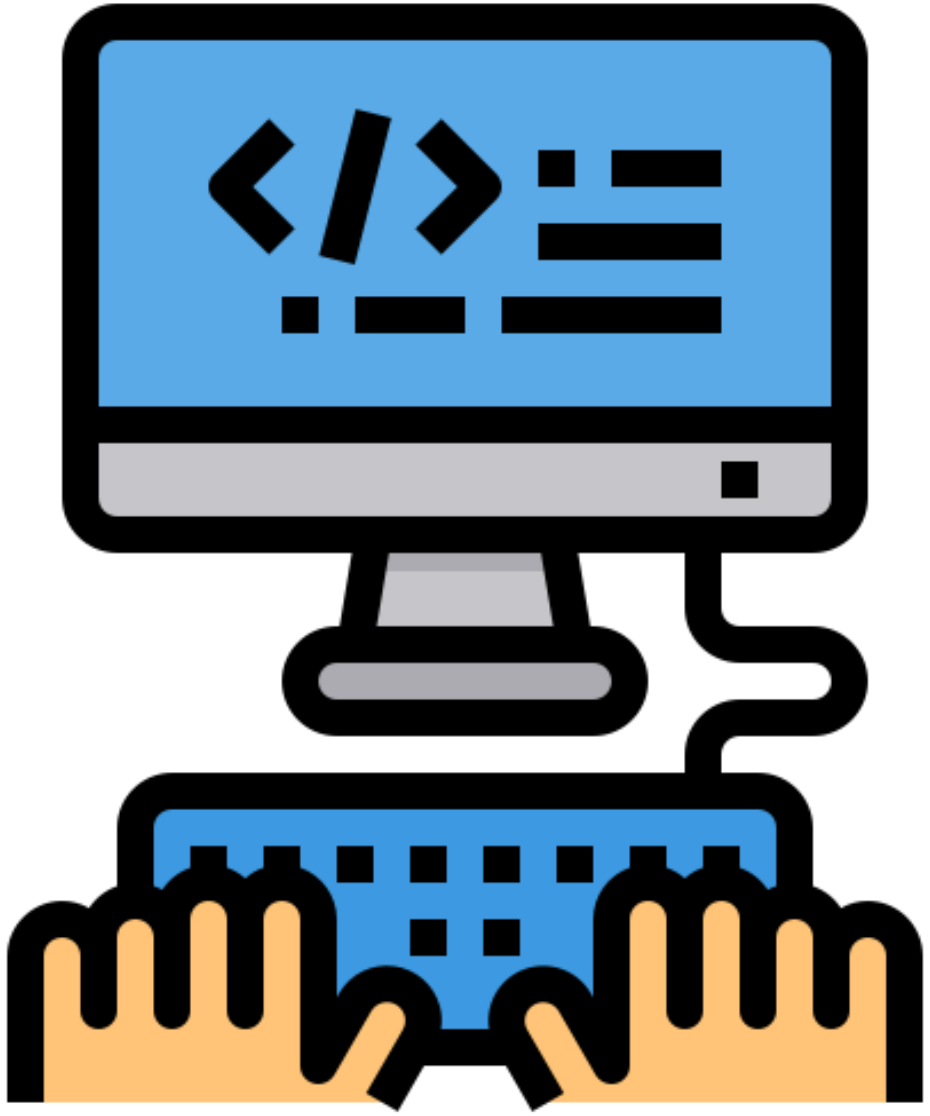
- Convert all letters to lowercase (check palindrome under case-insensitive criteria)
- Remove non-digit and non-letter characters.

```
//helper function
public String helper(String s){
    s = s.toLowerCase();
    String result = "";
    //int j = 0;
    //for loop
    for(int i = 0; i < s.length(); i++ ){
        if(Character.isDigit(s.charAt(i))|| Character.isLetter(s.charAt(i)))result += s.charAt(i);
    }
    return result;
}
```

```java
public class Palindrome
{
    //helper function
    public String helper(String s){
        s = s.toLowerCase();
        String result = "";
        //int j = 0;
        //for loop
        for(int i = 0; i < s.length(); i++ ){
            if(Character.isDigit(s.charAt(i))|| Character.isLetter(s.charAt(i)))result += s.charAt(i);
        }
        return result;
    }

    //isPalindrome function
    public boolean isPalindrome(String s){
        s = helper(s);
        // if loop
        if(s.length()== 0)return true;
        if(s.length()== 1)return true;
        if(s.length()== 2)return s.charAt(0)== s.charAt(1);
        return (s.charAt(0)== s.charAt(s.length()-1))&& isPalindrome(s.substring(1, s.length()-1));
    }

}
```

eC Learning Channel

```java
import java.util.Scanner;
public class PalindromeTest
{
    //main function
    public static void main(String[] args){
        //Scanner function
        Scanner input = new Scanner(System.in);
        boolean done = false;
        //while loop
        while(!done){
            System.out.println("Please enter a String or a Word: ");
            String word = input.nextLine();
            Palindrome rp = new Palindrome();
            System.out.println(rp.isPalindrome(word));
            System.out.println("Do you want to continue(Y/N)? ");
            String yes = input.nextLine();
            //if loop
            if(yes.length()!= 0 && (yes.charAt(0)=='Y' || yes.charAt(0)== 'y')) done = false;
            else done = true;
        }
    }

}
```



Recursion on 2D Grid

Section 5

Traverse a given Matrix using Recursion

Given a matrix mat[][] of size n x m, the task is to traverse this matrix using recursion.

Examples:

Input:
```
int[][] mat = {{1, 2, 3},
               {4, 5, 6},
               {7, 8, 9}};
```

Output : 1 2 3 4 5 6 7 8 9

Traverse a given Matrix using Recursion

Input:
```
int[][] mat = {{11, 12, 13},
               {14, 15, 16},
               {17, 18, 19}};
```

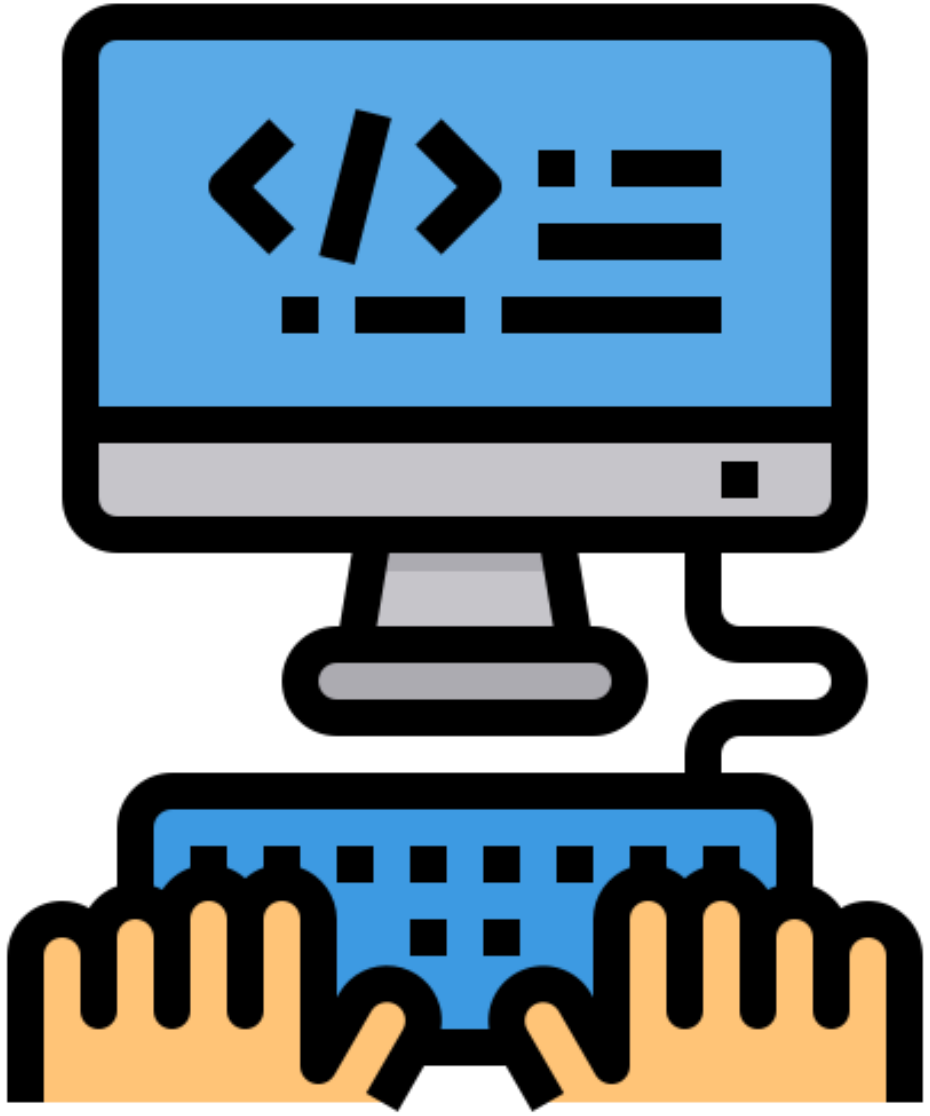
Output : `11 12 13 14 15 16 17 18 19`

Recursive Traversal Over 2D Array

```java
// Recursive function to traverse the matrix
static void traverse(int[][] mat, int i, int j) {
    // If the current position is the bottom-right
    // corner of the matrix
    if (i == mat.length - 1 && j == mat[0].length - 1) {
        System.out.println(mat[i][j]);
        return;
    }
    // Print the value at the current position
    System.out.print(mat[i][j] + " ");
    // If the end of the current row has not
      // been reached
    if (j < mat[0].length - 1) {

        // Move right
        traverse(mat, i, j + 1);
    }
    // If the end of the current column has been reached
    else if (i < mat.length - 1) {
        // Move down to the next row
        traverse(mat, i + 1, 0);
    }
}
```

Top Level: **GfG.java**

```
public static void main(String[] args) {
    int[][] mat = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} };
    traverse(mat, 0, 0);
}
```



Recursive Paradigm (Divide and Conquer, DP)

Section 6

Divide and Conquer Algorithm

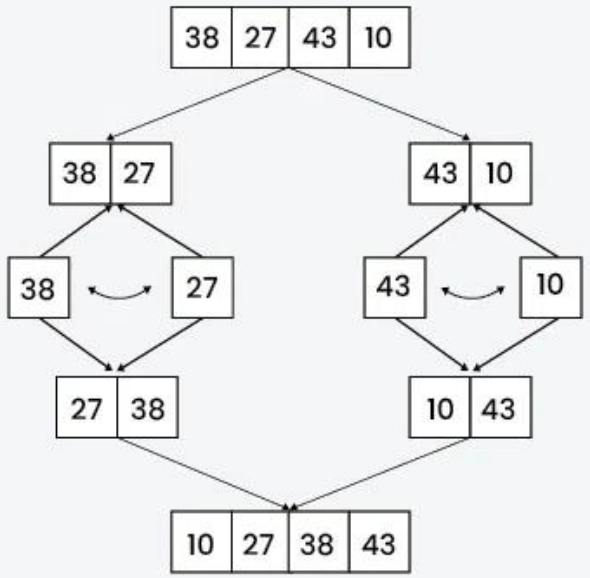
Divide and Conquer algorithm is a problem-solving strategy that involves.

- **Divide :** Break the given problem into smaller non-overlapping problems.
- **Conquer :** Solve Smaller Problems
- **Combine :** Use the Solutions of Smaller Problems to find the overall result.

Examples of Divide and Conquer are Merge Sort, Quick Sort, Binary Search and Closest Pair of Points.

Merge Sort

Divide and Conquer



Dynamic Programming or DP

- Dynamic Programming is an algorithmic technique with the following properties.
  - It is mainly an **optimization** over plain recursion. Wherever we see a recursive solution that has repeated calls for the same inputs, we can optimize it using **Dynamic Programming**.
  - The idea is to simply store the results of subproblems so that we do not have to re-compute them when needed later. **This simple optimization typically reduces time complexities from exponential to polynomial**.
  - Some popular problems solved using Dynamic Programming are **Fibonacci Numbers**, Diff Utility (Longest Common Subsequence), Bellman–Ford Shortest Path, Floyd Warshall, Edit Distance and Matrix Chain Multiplication.

```java
8  public class Fibonacii
9  {
10     public static int fibo(int n){
11         if (n<2) return 1;
12         return fibo(n-2) + fibo(n-1);
13     }
14
15     public static int fibodp(int n){
16         if (n<2) return 1;
17         int f=0;
18         int f1=1;
19         int f2=1;
20         int i=2;
21         while (i<=n){
22             f=f1 + f2;
23             f2 = f1;
24             f1 = f;
25             i++;
26         }
27         return f;
28     }
```

Top-Down Approach (Memoization):

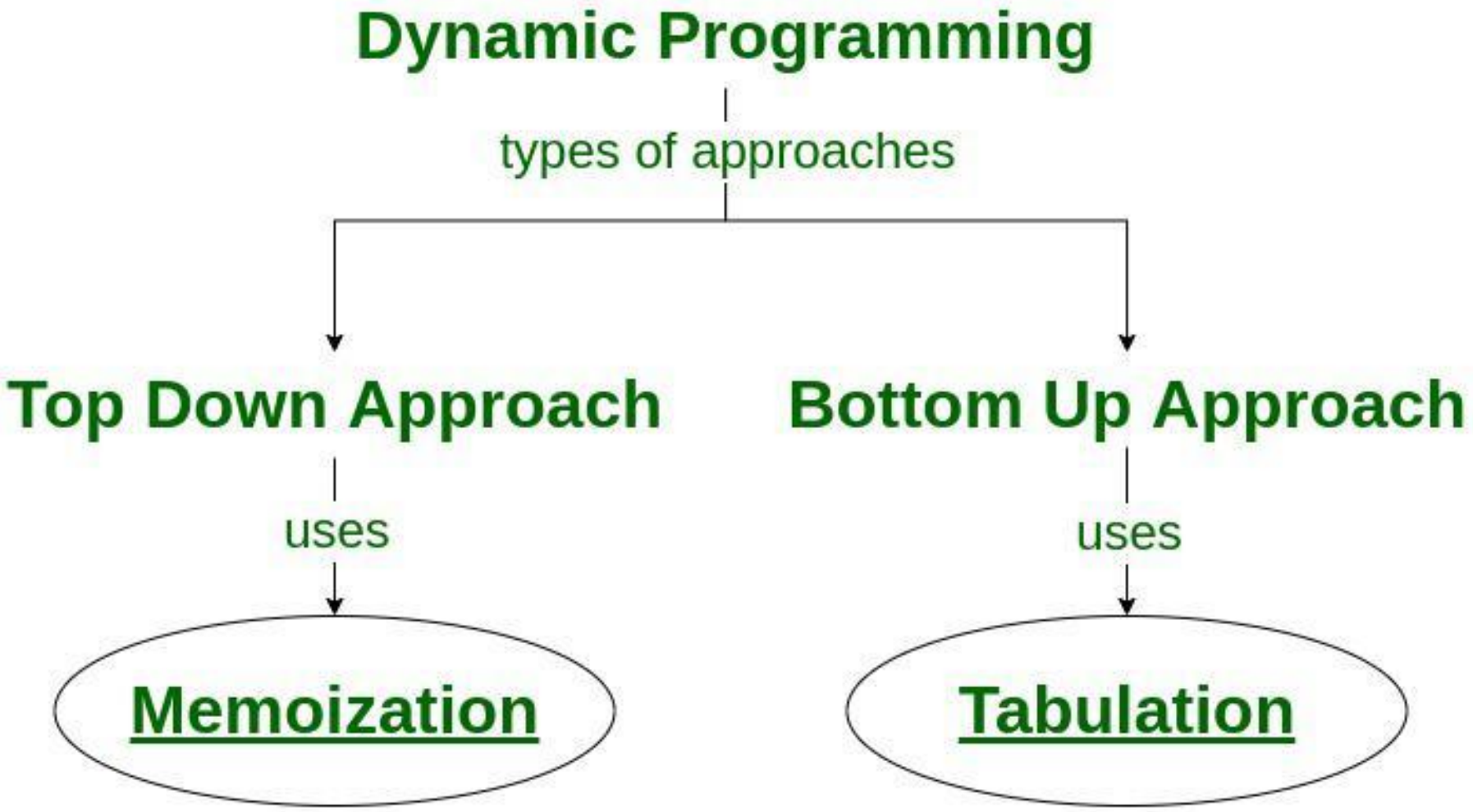
In the top-down approach, also known as **memoization**, we keep the solution recursive and add a memoization table to avoid repeated calls of same subproblems.

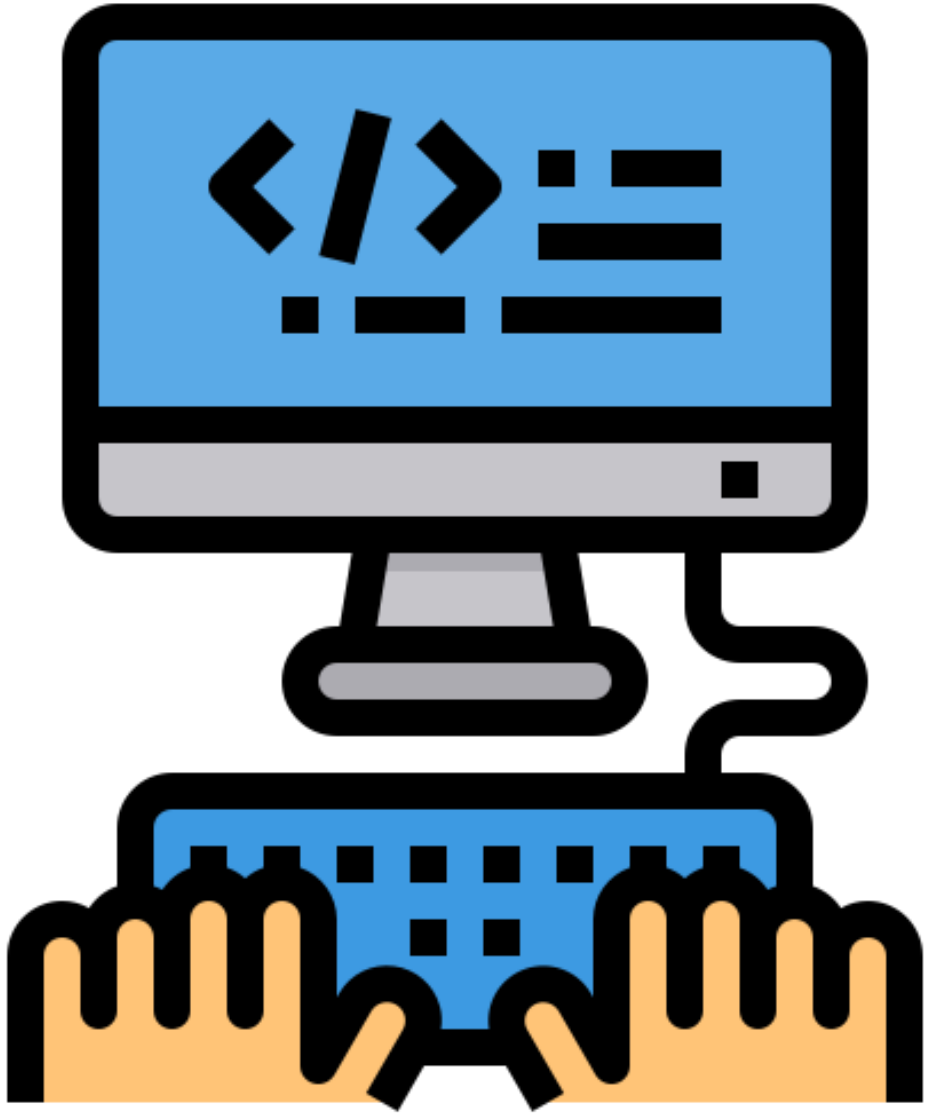
- Before making any recursive call, we first check if the memoization table already has solution for it.
- After the recursive call is over, we store the solution in the memoization table.

Bottom-Up Approach (Tabulation):

In the bottom-up approach, also known as **tabulation**, we start with the smallest subproblems and gradually build up to the final solution.

- We write an iterative solution (avoid recursion overhead) and build the solution in bottom-up manner.
- We use a dp table where we first fill the solution for base cases and then fill the remaining entries of the table using recursive formula.
- We only use recursive formula on table entries and do not make recursive calls.


Dynamic Programming
types of approaches
Top Down Approach
Bottom Up Approach
uses
uses
Memoization
Tabulation



Complexity Analysis

Section 7

Complexity Analysis for Binary Search

Number of Elements searched:

$$F(1) = 1$$

$$F(2) = F(1) + 1 + F(1) = 3$$

$$F(3) = F(2) + 1 + F(2) = 2 * F(2) + 1$$

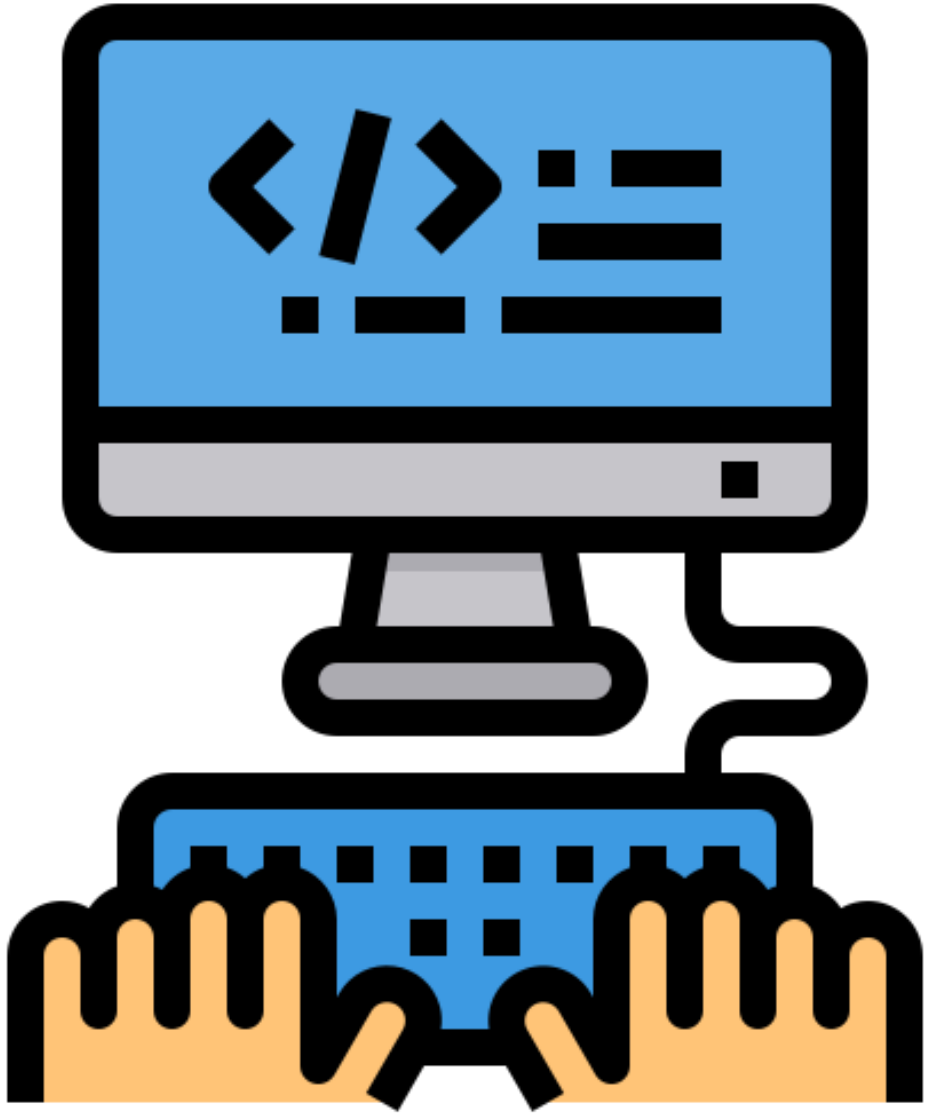
…

$$F(n) = 2^n - 1$$

Complexity Analysis for Fibonacci Numbers

Number of calls searched:

```
F(0) = 1
F(1) = 1
F(2) = F(1) + F(0) = 1 + 1 = 2
F(3) = F(2) + F(1) = 2 + 1 = 3
…
F(n) = F(n-1) + F(n-2)
```



Practice MCQ Questions

SECTION 8

Multiple Choice (1)

Consider the following recursive method:

```
public int mystery(int x){
    if (x==1) return 2;
    else return 2 * mystery(x-1);
}
```

What value is returned as a result of the call mystery(6)?

Multiple Choice (1)

Ⓐ 12

Ⓑ 32

Ⓒ 64

Ⓓ 128

Multiple Choice (1)

Ⓐ 12

Ⓑ 32

Ⓒ 64

Ⓓ 128

Multiple Choice (2)

Consider the following recursive method:

```
public int mystery(int x){
    if (x==0) return 0;
    else
      return (x + mystery(x/2)+mystery(x/4));
}
```

What value is returned as a result of the call mystery(10)?

Multiple Choice (1)

Ⓐ 12

Ⓑ 20

Ⓒ 22

Ⓓ 35

Multiple Choice (1)

Ⓐ 12

Ⓑ 20

Ⓒ 22

Ⓓ 35

Multiple Choice (3)

Consider the following non-recursive method:

```
// precondition: x >=0
public static int mystery(int x){
    int sum = 0;
    while (x >=0){
        sum += x;
        x--;
    }
    return sum;
}
```

Multiple Choice (3)

Which of the following recursive methods are equivalent to the method above?

I.
```
public int mystery2(int x){
    if (x==0) return 0;
    return (x + mystery2(x-1));
}
```

II.
```
public int mystery3(int x){
    if (x==0) return 0;
    return mystery3(x-1);
}
```

Multiple Choice (3)

Which of the following recursive methods are equivalent to the method above?

Ⅲ.
```
public int mystery4(int x){
    if (x==1) return 1;
    return (x + mystery4(x-1));
}
```

Multiple Choice (3)

Ⓐ Ⅰ only

Ⓑ Ⅲ only

Ⓒ Ⅰ and Ⅱ

Ⓓ Ⅱ and Ⅲ

Multiple Choice (3)

Ⓐ Ⅰ only

Ⓑ Ⅲ only

Ⓒ Ⅰ and Ⅱ

Ⓓ Ⅱ and Ⅲ

Multiple Choice (4)

Consider the following method:

```
public int mystery(int x, int y){
    if (x>=100 || y<=0) return 1;
    else return mystery(x+10, y-3);
}
```

What value is returned by the call `mystery(30, 18)`?

Multiple Choice (4)

Ⓐ 0

Ⓑ 1

Ⓒ 6

Ⓓ Nothing will be returned due to infinite recursion.

Multiple Choice (4)

Ⓐ 0

❷ 1

Ⓒ 6

Ⓓ Nothing will be returned due to infinite recursion.

Multiple Choice (5)

Consider the following method:

```
public int mystery(int x){
    if (x <= 1) return 1;
    else return /*missing code*/ ;
}
```

Which of the following could be used to replace /* missing code */ so that the value of `mystery(10)` is `32`?

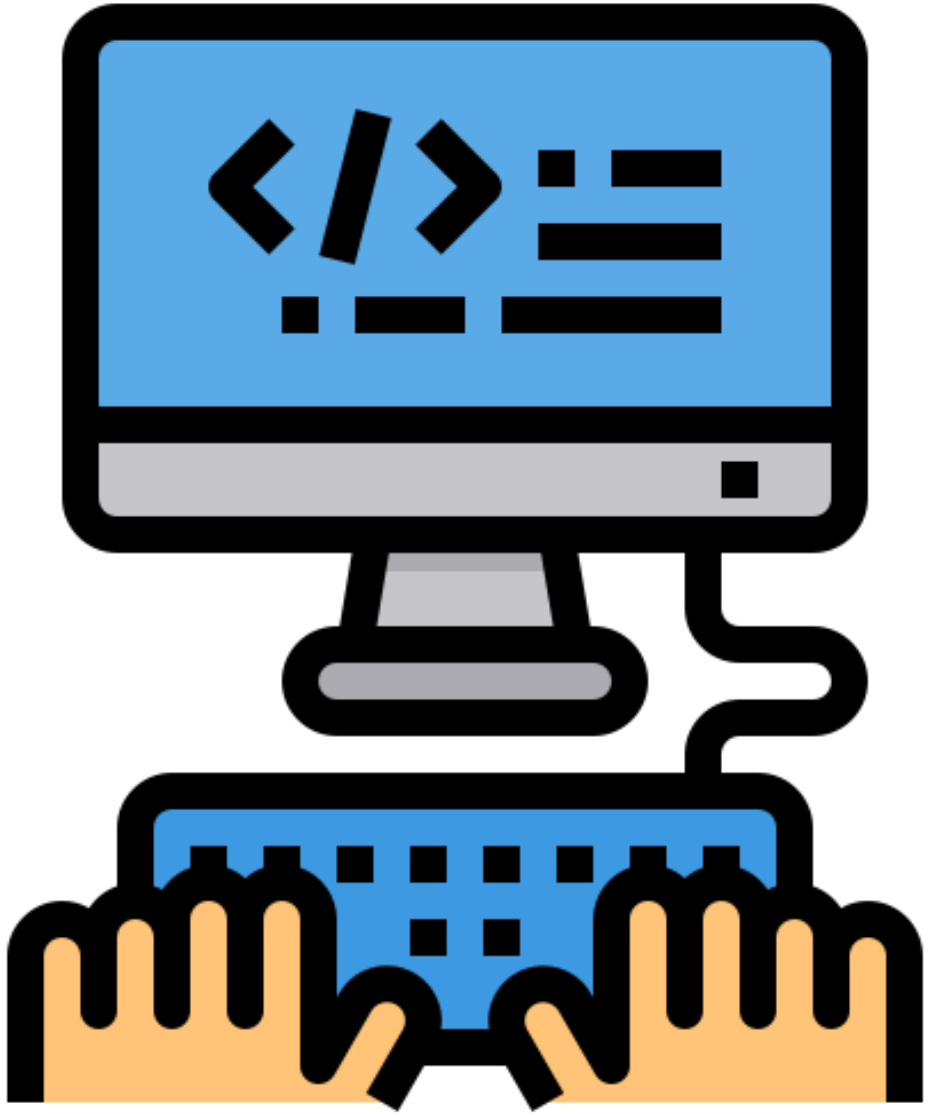
Multiple Choice (5)

Ⓐ `mystery(x-1) + mystery(x-2)`

Ⓑ `2 * mystery(x-2)`

Ⓒ `2 * mystery(x-1)`

Ⓓ `4 * mystery(x-4)`



Recursive Algorithms

SECTION 9

GCD Subtraction Version

```
public int gcd(int m, int n){
    if (m==0) return n;
    if (n==0) return m;
    if (m>=n) gcd(m-n, n);
    return gcd(m, n-m);
}
```

GCD Modulus Version

```
public int gcd(int m, int n){
    if (n==0) return m;
    return gcd(n, m%n);
}
```

Palindrome Check

```
public boolean isPalindrome(String s){
    if (s.length()<2) return true;
    if (s.charAt(0) != s.charAt(s.length()-1))
        return false;
    return isPalindrome(s.substring(1,
                                s.length()-1));
}
```

Binary Search

```
public int search(int[] a, int key){
    return search(a, key, 0, a.length-1);
}
public int search(int[] a, int key, int low,
                  int high){
    if (low > high) return -1-low;
    int mid = (low+high)/2;
    if (a[mid]==key) return mid;
    if (a[mid]>key) return search(a, key, low,
                                  mid-1);
    return search(a, key, mid+, high);
}
```

Merge Sort

```
public int[] merge(int[] a, int[] b){
   int p=0, q=0, r=0;
   int[] c = new int[a.length+b.length];
   while (p<a.length && q<b.length){
      if (a[p]<=b[q]) c[r++] = a[p++];
      else c[r++] = b[q++];
   }
   while(p<a.length) c[r++] = a[p++];
   while(q<b.length) c[r++] = b[q++];
   return c;
}
```

Merge Sort

```
public int[] sort(int[] a){
  if (a.length<2) return a;
  int low = 0, high = a.length-1;
  int mid = (low+high)/2;
  int[] first = new int[mid+1];
  int[] second = new int[a.length-first.length];
  int p = 0;
  for (int i=0; i<first.length; i++) first[i]=a[p++];
  for (int i=0; i<second.length; i++ second[i]=a[p++];
  first = sort(first);
  second = sort(second);
  int[] third = merge(first, second);
  return third;
}
```